

Functional app checklist

Native Academy, community, field tools, member access, web release, and mobile delivery

DEFINITION OF DONE

Tomorrow's finish line

An active Dirty Turf member can sign into the native app, open imported Academy lessons, use the community and events, measure a turf job, calculate exact infill bags and customer price, save the result, and reopen it on a second device. Production web and test-store builds are verified with client-owned accounts.

APP EXPERIENCE

Native Academy, lessons, quizzes, community, events, member directory, field tools, and mobile navigation are built.

CONTENT ARCHIVE

One course, 21 module groups, 128 lessons, 137 assets, and 3 quizzes are captured outside Git.

BACKEND READY

Supabase schema, RLS, private storage, importer, magic-link accounts, Stripe billing, and source-aware entitlements are

NATIVE DELIVERY

Capacitor iOS simulator and Android debug builds pass. Real-device AR and signed store builds remain release gates.

Verified build state

-  Schema contracts cover all 50 public tables and 10 migrations; TypeScript, 53 tests, the PWA asset manifest, and the production build pass.
-  The infill calculator passes a live 20 x 18 ft test: 360 sq ft, 180 lb at 0.50 lb/sq ft, 5 forty-pound bags, 4 fifty-pound bags, and \$259.20 at \$0.72/sq ft.
-  Mobile QA passes from 320 x 568 through tablet with no horizontal overflow; the signed-out Magic Link gate remains mounted after the startup watchdog.
-  Unsigned iOS simulator compilation, Android debug assembly, Android unit tests, and Android lint succeed.
-  The private HighLevel course archive validates and remains ignored by Git.
-  Production Supabase migration, Stripe test-mode proof, member import/provisioning, SMTP delivery, deep-link proof, real-device AR, signed store builds, and production smoke tests still require client accounts or devices.

NOT DONE UNTIL

Do not retire HighLevel or invite members until archive counts reconcile, the import dry run passes, client SMTP delivers, and active/cancelled access rules are proven.

Client-owned accounts first

The only steps that require the client to sign in, verify identity, or supply billing

8:00 AM

Finish account-owner actions

OWNER: Dirty Turf owner

- ☐ Confirm GitHub repository stevedifabio-oss/Dirty-Turf, the active Netlify site, and Supabase project ipbtldajgsxixqmajec are owned by Dirty Turf LLC admins.
- ☐ Supabase: configure a client-owned SMTP sender, verify its DNS records, set the Site URL to the exact production HTTPS origin, and allow that origin plus `com.dirtyturf.academy://auth/callback`.
- ☐ Apple: add the renewal card, accept pending agreements, confirm App Store Connect access, and keep Dirty Turf LLC as the legal seller.
- ☐ Google Play: finish organization identity, website, and phone verification. App publication stays blocked until Google approves the developer account.
- ☐ Confirm access to `dirtyturf.com` DNS plus the privacy, support, and account-deletion URLs needed by both stores.
- ☐ After the final HighLevel archive capture, rotate the private integration token shared in chat and store the replacement only as a server secret.
- ☐ Use only client-owned email, recovery, MFA, billing, and password-manager records. Do not connect a consultant's personal account or terminal credentials.

Accounts required

HighLevel, Supabase, GitHub, Netlify, domain/DNS, transactional SMTP, Apple Developer/App Store Connect, Google Play Console/Google Cloud, and a client-owned Stripe account before payments are enabled.

Secret boundary

```
# Browser-safe (Netlify)
VITE_SUPABASE_URL=...
VITE_SUPABASE_PUBLISHABLE_KEY=...
VITE_STRIPE_ACADEMY_PAYMENT_LINK=https://buy.stripe.com/...

# Server-only (Supabase secrets)
SUPABASE_SERVICE_ROLE_KEY=...
GHL_PRIVATE_INTEGRATION_TOKEN=...
GHL_LOCATION_ID=...
APP_URL=... APP_ALLOWED_ORIGINS=... AUTH_REDIRECT_URLS=...
STRIPE_SECRET_KEY=... STRIPE_WEBHOOK_SECRET=...
```

- ☐ Never put the HighLevel token or Supabase service-role key in a `VITE_` variable, browser storage, app bundle, screenshot, PDF, or Git commit.
- ☐ Record one client owner and one recovery owner for every account before launch.

NOT DONE UNTIL

Every account is client-owned, every secret has a rotation path, and no personal credential or billing profile is attached to the product.

Activate the Supabase backend

Apply the reviewed schema, deploy the importer, and prove authorization before content import

9:00 AM

Migrate and deploy

OWNER: Backend owner

```
npm run check
npx supabase link --project-ref ipbtldajgsoxixqmajec
npx supabase db push
npx supabase functions deploy academy-import
npx supabase functions deploy academy-invite-members
npx supabase functions deploy create-checkout
npx supabase functions deploy create-billing-portal
npx supabase functions deploy stripe-webhook --no-verify-jwt
npx supabase functions deploy health
```

- ☐ Apply all ten ordered migrations through 20260918080238_academy_billing_entitlements.sql and confirm Academy, community, billing, entitlement, invite, asset, progress, and import-ledger tables exist.
- ☐ In Supabase API settings, expose only the intended schema. Re-check RLS, grants, revoked anon/public access, and security-invoker views after any exposure change.
- ☐ Create one Academy owner/admin and one normal active member. Verify the importer rejects the member and accepts only the owner/admin session.
- ☐ Create two company organizations. Prove company B cannot read company A properties, calculations, visits, photos, or storage objects.
- ☐ Confirm academy-assets is private. Test an authorized signed asset URL and an unauthorized read of the same object.
- ☐ Confirm claim_academy_memberships is authenticated-only and ensure_academy_user_workspace is service-role-only. A normal member must not call the provisioning Edge Function.
- ☐ Configure client SMTP and exact auth redirects before notifications. Test web and native PKCE links from cold start and while the app is open, plus logout, expiry, and second-device login.
- ☐ Enable backups, document the restore owner, and record the migration version plus Edge Function versions.

Backend acceptance record

Migration applied	OWNER / TIME / RESULT
Cross-tenant RLS denied	OWNER / TIME / RESULT
Private Academy and photo storage proven	OWNER / TIME / RESULT
SMTP and auth redirects proven	OWNER / TIME / RESULT

NOT DONE UNTIL

Stop if a service-role key appears in the browser bundle, if any cross-company record is readable, or if a normal member can run an admin import.

Import the HighLevel archive

The private source package is composed and validated; reconcile it in production before any invite is sent

11:00 AM

Dry run, reconcile, then import

OWNER: Migration owner

Private archive ready

- ☒ 1 course, 21 normalized module groups, 128 unique lessons, 137 assets, and 3 quizzes are stored in output/private/ghl-course-import.json outside Git.
- ☒ 60 login-eligible members plus one historical non-login author, 49 enrollments, 15 categories, 62 posts, 59 comments, and 5 events are reconciled.
- ☒ Lesson status is preserved: 109 published and 19 drafts. Quiz questions, answer options, explanations, and only verifiable answer keys are supported.
- ☒ The composed import manifest validates with SHA-256
0e1f53311b4635a7ed2969ff5bf6e2b038781eeb3126a958e3a82633f1aee.
- ☒ Source course percentages are preserved as an aggregate floor; the import does not fabricate lesson completions, reaction identities, or RSVP identities.

Production import steps

- ☐ Create the client owner in Supabase, copy the owner organization UUID into the ignored manifest, and validate it again.
- ☐ Run academy:access-audit. Require 60 unique login emails, 49 enrolled members, zero duplicates, and allEnrolledCanLogin:true.
- ☐ Invoke academy-import with commit:false. Match every returned count to the private report before allowing a write.
- ☐ Invoke academy-import with commit:true once. Re-run it to prove provider IDs make the operation idempotent.
- ☐ Open at least three lessons, posts, comments, events, member profiles, assets, and enrollment states in both HighLevel and the native app.
- ☐ Confirm ownership and long-term availability for every Filesafe, image, video, PDF, certificate, and download URL. Copy owned assets to private storage when permitted.

Import sequence

```
npm run academy:access-audit
academy-import { commit:false } -> reconcile counts
academy-import { commit:true } -> idempotent import
member access preview/provision -> readiness, no email
```

- ☐ Provision until all 60 current members report allEligibleReady:true and all 49 enrollments are linked. Accounts use Magic Links; passwords do not migrate.
- ☐ Prove a source member can sign in and an unknown email cannot create an account. Test owner, admin, moderator, active, cancelled, suspended, and unassigned access.

NOT DONE UNTIL

Do not commit private manifests, member email addresses, course bodies, or community exports to Git. Keep the archive encrypted and client-owned.

Activate web payments

Stripe-hosted checkout grants Supabase access; native apps remain consumption-only

1:00 PM

Configure Stripe and prove access

OWNER: Billing owner

- ☐ Create a client-owned Stripe account, complete business verification, add Dirty Turf administrators, require MFA, and configure client billing plus recovery ownership.
- ☐ In Stripe test mode, create the Academy product and recurring or one-time Price. Do not accept a price or amount from the app browser.
- ☐ Insert the Price ID into an inactive academy_billing_plans row, map included courses in academy_billing_plan_courses, verify the community-access choice, then activate the plan.
- ☐ Register the stripe-webhook Edge Function for checkout.session.completed, checkout.session.async_payment_succeeded, and subscription created, updated, and deleted events.
- ☐ Configure the Stripe Customer Portal and a web Payment Link. Redirect successful payment to `https://YOUR_APP_DOMAIN/?checkout=success` and set `VITE_STRIPE_ACADEMY_PAYMENT_LINK` in Netlify.
- ☐ Complete an authenticated web Checkout. Verify customer, subscription/payment, access grant, course enrollment, and processed integration event rows.
- ☐ Complete a Payment Link purchase with a new email. Verify Supabase silently creates the confirmed account, member, provisioned invite, and grants; then sign in through Magic Link.
- ☐ Replay the same webhook and prove it is idempotent. Force one processing failure and prove the unprocessed event retries instead of being discarded.
- ☐ Cancel a Stripe-only test member and confirm access is removed at the intended time. Cancel a Stripe record for an imported member and confirm the separate HighLevel import grant preserves access.
- ☐ Open Billing Portal on the web. Confirm the iOS and Android bundles show no purchase button or external checkout link and only consume access bought elsewhere.

Billing acceptance

Test Checkout and Payment Link	OWNER / TIME / RESULT
Webhook replay and recovery	OWNER / TIME / RESULT
Grant and cancellation rules	OWNER / TIME / RESULT
Native consumption-only review	OWNER / TIME / RESULT

NOT DONE UNTIL

Do not switch Stripe to live mode until test checkout, Magic Link access, cancellation, webhook retry, and imported-member preservation all pass.

1:00 PM

Run the measured-yard test

OWNER: Field QA owner

- ☒ Manual mode accepts length and width, calculates area correctly, and preserves decimal precision before display rounding.
- ☒ Application rate is a dropdown from 0.25 through 3.00 lb/sq ft. Bag price, bag coverage, cleaning plan, and property name are removed from the calculator.
- ☒ Total pounds equals exact area times selected rate; 40-lb and 50-lb bag counts round upward independently; customer price uses charge per sq ft.
- ☒ Native iOS ARKit and Android ARCore point-placement bridges are implemented, and map tracing remains a separate measurement path.
- ☐ On a supported iPhone, place at least four live AR points around a tape-measured yard, undo one point, finish, and compare area plus perimeter.
- ☐ Repeat on a supported Samsung/Android phone using ARCore. Record model, OS, tracking quality, measured area, and error percentage.
- ☐ Trace the same yard on the map, test multiple turf polygons, and verify the combined area against the known reference.
- ☐ Deny camera and location permissions; test unsupported AR hardware, tracking loss, rotation, background/resume, offline mode, and duplicate save protection.
- ☐ Save a calculation with photos, refresh, sign in on a second device, and confirm exact area, rate, pounds, both bag counts, price, method, date, and history.

Reference calculation

20 ft x 18 ft	= 360 sq ft
360 sq ft x 0.50 lb/sq ft	= 180 lb
ceil(180 / 40) / ceil(180 / 50)	= 5 / 4 bags
360 sq ft x \$0.72/sq ft	= \$259.20

NOT DONE UNTIL

Live camera measurement is not approved by a simulator build. It must place real points and match a tape-measured job on both mobile platforms.

3:30 PM

Ship the web release

OWNER: Release owner

```

npm run check
npm run academy:access-audit
npm run academy:validate -- output/private/dirty-turf-academy-import.json
npm run native:sync
git diff --check
git status --short

```

- ☐ Run a secret scan and confirm .env.local plus output/private remain ignored. Review the complete diff before commit.
- ☐ Push the release branch, open or update the pull request, require checks, merge to main, and record the commit SHA.
- ☐ Set VITE_SUPABASE_URL, the Supabase publishable key, and the public buy.stripe.com Payment Link in Netlify. Keep service-role, HighLevel, and Stripe secret keys in server-side Supabase settings.
- ☐ Deploy a Netlify preview first. Verify SPA routes, auth callbacks, manifest/service worker, icons, security headers, and no stale duplicate site is receiving production traffic.
- ☐ Verify 320, 360, 390, and 430 px widths plus landscape, tablet, and desktop. Check calculator keyboard behavior, safe areas, bottom navigation, long lesson content, quizzes, community, and events.
- ☐ Deploy production and re-run sign-in, Academy, lesson progress, quiz, post, comment, reaction, bookmark, event RSVP, calculator save, photo, history, Checkout, webhook, and Billing Portal flows.
- ☐ Inspect production console and network errors. Confirm no private token, member archive, service-role key, or source export is downloadable.
- ☐ Record Netlify deploy ID, Supabase migration and function versions, production URL, smoke-test account, rollback owner, and rollback steps.

Production acceptance

GitHub checks and merge

OWNER / TIME / RESULT

Netlify preview and production smoke

OWNER / TIME / RESULT

Supabase logs clean

OWNER / TIME / RESULT

NOT DONE UNTIL

A successful build or deploy status is not production proof. Repeat the golden path on the deployed URL with an authenticated client test account.

Mobile stores and cutover

Create signed test builds, invite a small cohort, and keep a reversible HighLevel handoff

5:00 PM

Build, invite, and cut over

OWNER: Product owner

- ☐ In Xcode, confirm bundle ID, Dirty Turf team, automatic signing, iOS 15 minimum, icons, splash, camera/location descriptions, and archive a Release build to TestFlight.
- ☐ In Android Studio, confirm application ID, version code/name, signing key ownership, icons, permissions, privacy declarations, and create a signed AAB for Internal testing.
- ☐ Add store name, description, category, screenshots, privacy policy, support URL, account-deletion URL, reviewer login, data-safety/privacy answers, and content-rating responses.
- ☐ Test install, update, deep link, magic link, logout, expired/reused links, camera, location, photo upload, map, lesson, community, and notification behavior on physical devices.
- ☐ Verify store-review builds are consumption-only: no native purchase CTA or web checkout link. Supply a reviewer account with pre-existing Academy access.
- ☐ Require the access preview to show all 60 current members ready and all 49 enrollments linked. Notify a 3-5 person pilot, verify delivery and entitlements, then notify remaining members in batches of at most 25.
- ☐ Freeze HighLevel content changes, take one final delta capture, re-run the import dry run, reconcile counts, and commit the approved delta import.
- ☐ Keep HighLevel read-only as the rollback source until Dirty Turf approves the native archive, member access, and production behavior. Retain the encrypted export independently.
- ☐ Monitor authentication, invite failures, Edge Function errors, database errors, app crashes, and member support during the first 24 hours.

Final cutover order

1. Freeze GHL. 2. Capture delta. 3. Validate. 4. Dry run. 5. Commit import. 6. Provision all accounts. 7. Pilot links. 8. Notify remaining members. 9. Production smoke. 10. Client approval.

Final sign-off

Content and member reconciliation	OWNER / TIME / RESULT
Security, privacy, and ownership	OWNER / TIME / RESULT
iPhone and Android field workflow	OWNER / TIME / RESULT
Web production and store test builds	OWNER / TIME / RESULT
Cutover and rollback approval	OWNER / TIME / RESULT

NOT DONE UNTIL

Launch only when every sign-off has an owner, timestamp, evidence link, and explicit pass. Otherwise keep HighLevel live and the native app in labeled pilot mode.